

Scallop: A Language for Neurosymbolic Programming

ZIYANG LI*, University of Pennsylvania

JIANI HUANG*, University of Pennsylvania

MAYUR NAIK, University of Pennsylvania

We present Scallop, a language which combines the benefits of deep learning and logical reasoning. Scallop enables users to write a wide range of neurosymbolic applications and train them in a data- and compute-efficient manner. It achieves these goals through three key features: 1) a flexible symbolic representation that is based on the relational data model; 2) a declarative logic programming language that is based on Datalog and supports recursion, aggregation, and negation; and 3) a framework for automatic and efficient differentiable reasoning that is based on the theory of provenance semirings. We evaluate Scallop on a suite of eight neurosymbolic applications from the literature. Our evaluation demonstrates that Scallop is capable of expressing algorithmic reasoning in diverse and challenging AI tasks, provides a succinct interface for machine learning programmers to integrate logical domain knowledge, and yields solutions that are comparable or superior to state-of-the-art models in terms of accuracy. Furthermore, Scallop’s solutions outperform these models in aspects such as runtime and data efficiency, interpretability, and generalizability.

1 INTRODUCTION

Classical algorithms and deep learning embody two prevalent paradigms of modern programming. Classical algorithms are well suited for exactly-defined tasks, such as sorting a list of numbers or finding a shortest path in a graph. Deep learning, on the other hand, is well suited for tasks that are not tractable or feasible to perform using classical algorithms, such as detecting objects in an image or parsing natural language text. These tasks are typically specified using a set of input-output training data, and solving them involves learning the parameters of a deep neural network to fit the data using gradient-based methods.

The two paradigms are complementary in nature. For instance, a classical algorithm such as the logic program P depicted in Figure 1a is interpretable but operates on limited (e.g., structured) input r . In contrast, a deep neural network such as M_θ depicted in Figure 1b can operate on rich (e.g., unstructured) input x but is not interpretable. Modern applications demand the capabilities of both paradigms. Examples include question answering [Rajpurkar et al. 2016], code completion [Chen et al. 2021b], and mathematical problem solving [Lewkowycz et al. 2022], among many others. For instance, code completion requires deep learning to comprehend programmer intent from the code context, and classical algorithms to ensure that the generated code is correct. A natural and fundamental question then is how to program such applications by integrating the two paradigms.

Neurosymbolic programming is an emerging paradigm that aims to fulfill this goal [Chaudhuri et al. 2021]. It seeks to integrate symbolic knowledge and reasoning with neural architectures for better efficiency, interpretability, and generalizability than the neural or symbolic counterparts alone. Consider the task of handwritten formula evaluation [Li et al. 2020], which takes as input a formula as an image, and outputs a number corresponding to the result of evaluating it. An input-output example for this task is $\langle x = \mathcal{A} \div \mathcal{B} \div \mathcal{C}, y = 1.6 \rangle$. A neurosymbolic program for this task, such as the one depicted in Figure 1c, might first apply a convolutional neural network M_θ to the input image to obtain a structured intermediate form r as a sequence of symbols ['1', '+', '3', '/', '5'], followed by a classical algorithm P to parse the sequence, evaluate the parsed formula, and output the final result 1.6.

*The two authors contributed equally on the paper

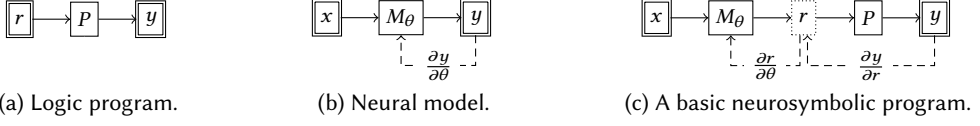


Fig. 1. Comparison of different paradigms. Logic program P accepts only structured input r whereas neural model M_θ with parameter θ can operate on unstructured input x . Supervision is provided on data indicated in double boxes. Under *algorithmic supervision*, a neurosymbolic program must learn θ without supervision on r .

Despite significant strides in individual neurosymbolic applications [Chen et al. 2020; Li et al. 2020; Mao et al. 2019; Minervini et al. 2020; Wang et al. 2019; Yi et al. 2018], there is a lack of a language with compiler support to make the benefits of the neurosymbolic paradigm more widely accessible. We set out to develop such a language and identified five key criteria that it should satisfy in order to be practical. These criteria, annotated by the components of the neurosymbolic program in Figure 1c, are as follows:

- (1) A symbolic data representation for r that supports diverse kinds of data, such as image, video, natural language text, tabular data, and their combinations.
- (2) A symbolic reasoning language for P that allows to express common reasoning patterns such as recursion, negation, and aggregation.
- (3) An automatic and efficient differentiable reasoning engine for learning $(\frac{\partial y}{\partial r})$ under *algorithmic supervision*, i.e., supervision on observable input-output data (x, y) but not r .
- (4) The ability to tailor learning $(\frac{\partial y}{\partial r})$ to individual applications’ characteristics, since non-continuous loss landscapes of logic programs hinder learning using a one-size-fits-all method.
- (5) A mechanism to leverage and integrate with existing training pipelines $(\frac{\partial r}{\partial \theta})$, implementations of neural architectures and models M_θ , and hardware (e.g., GPU) optimizations.

In this paper, we present Scallop, a language which satisfies the above criteria. The key insight underlying Scallop is its choice of three inter-dependent design decisions: a relational model for symbolic data representation, a declarative language for symbolic reasoning, and a provenance framework for differentiable reasoning. We elaborate upon each of these decisions.

Relations can represent arbitrary graphs and are therefore well suited for representing symbolic data in Scallop applications. For instance, they can represent *scene graphs*, a canonical symbolic representation of images [Johnson et al. 2015], or abstract syntax trees, a symbolic representation of natural language text. Further, they can be combined in a *relational database* to represent multi-modal data. Relations are also a natural fit for probabilistic reasoning [Dries et al. 2015], which is necessary since symbols in r produced by neural model M_θ can have associated probabilities.

Next, the symbolic reasoning program P in Scallop is specified in a declarative logic programming language. The language extends Datalog [Abiteboul et al. 1995] and is expressive enough for programmers to specify complex domain knowledge patterns that the neural model M_θ would struggle with. Datalog implementations can take advantage of optimizations from the literature on relational database systems. This in turn enables efficient inference and learning since the logical domain knowledge specifications of the task at hand help reduce the burden of M_θ , whose responsibilities are now less complex and more modular. Finally, Datalog is rule-based, which makes programs easier to write, debug, and verify. It also facilitates inferring them by leveraging techniques from program synthesis [Gulwani et al. 2017] and ILP [Cropper et al. 2022].

While symbolic reasoning offers many aforementioned benefits, it poses a fundamental challenge for learning parameter θ . Deep learning relies on gradient-based methods, enabled by the differentiable nature of the low-level activation functions that comprise M_θ , to obtain $\frac{\partial r}{\partial \theta}$. The key challenge

then concerns how to support automatic and efficient differentiation of the high-level logic program P to obtain $\frac{\partial y}{\partial r}$, which can be used in conjunction with $\frac{\partial r}{\partial \theta}$ to compute $\frac{\partial y}{\partial \theta}$. Scallop addresses this problem by leveraging the framework of *provenance semirings* [Green et al. 2007]. The framework proposes a common algebraic structure for applications that define annotations (i.e., tags) for tuples and propagate the annotations from inputs to outputs of relational algebra (RA) queries. One of our primary contributions is a novel adaptation of the framework for differentiable reasoning for an extended fragment of RA that includes recursion, negation, and aggregation. Scallop implements an extensible library of provenance structures including the extended max-min semiring and the top- k proofs semiring [Huang et al. 2021]. We further demonstrate that different provenance structures enable different heuristics for the gradient calculations, providing an effective mechanism to tailor the learning process to individual applications’ characteristics.

We have implemented a comprehensive and open-source toolchain for Scallop in 45K LoC of Rust. It includes a compiler, an interpreter, and PyTorch bindings to integrate Scallop programs with existing machine learning pipelines. We evaluate Scallop using a suite of eight neurosymbolic applications that span the domains of image and video processing, natural language processing, planning, and knowledge graph querying, in a variety of learning settings such as supervised learning, reinforcement learning, rule learning, and contrastive learning. Our evaluation demonstrates that Scallop is expressive and yields solutions of comparable, and often times superior, accuracy than state-of-the-art models. We show additional benefits of Scallop’s solutions in terms of runtime and data efficiency, interpretability, and generalizability.

Any programming language treatise would be remiss without acknowledging the language’s lineage. TensorLog [Cohen et al. 2017] and DeepProbLog (DPL) [Manhaeve et al. 2021] pioneered the idea of extending probabilistic logic programming languages (e.g., ProbLog [Dries et al. 2015]) with differentiable reasoning. Scallop was originally proposed in [Huang et al. 2021] to improve the scalability of DPL by using Datalog instead of Prolog and relaxing its exact probabilistic semantics. We build upon [Huang et al. 2021] by extending its expressiveness, formalizing the semantics, developing a customizable provenance framework, and providing a practical toolchain.

The rest of the paper is organized as follows. Section 2 presents an illustrative overview of Scallop. Section 3 describes Scallop’s language for symbolic reasoning. Section 4 presents the differentiable reasoning framework. Section 5 describes our implementation of Scallop. Section 6 empirically evaluates Scallop on a benchmark suite. Section 7 surveys related work and Section 8 concludes.

2 ILLUSTRATIVE OVERVIEW

We illustrate Scallop using an reinforcement learning (RL) based planning application which we call PacMan-Maze. The application, depicted in Fig. 2a, concerns an intelligent agent realizing a sequence of actions in a simplified version of the PacMan maze game. The maze is an implicit 5×5 grid of cells. Each cell is either empty or has an entity, which can be either the *actor* (PacMan), the *goal* (flag), or an *enemy* (ghost). At each step, the agent moves the actor in one of four directions: up, down, right, or left. The game ends when the actor reaches the goal or hits an enemy. The maze is provided to the agent as a raw image that is updated at each step, requiring the agent to process sensory inputs, extract relevant features, and logically plan the path to take. Additionally, each session of the game has randomized initial positions of the actor, the goal, and the enemies.

Concretely, the game is modeled as a sequence of interactions between the agent and an environment, as depicted in Fig. 2b. The game state $s_i \in S$ at step i is a 200×200 colored image ($S = \mathbb{R}^{200 \times 200 \times 3}$). The agent proposes an action $a_i \in A = \{\text{up, down, right, left}\}$ to the environment, which generates a new image s_{i+1} as the next state. The environment also returns a reward r_i to